Name : Arian Ahmed

Student ID : 20230932

Site : http://172.10.9.70/project/

Disclaimer: The frontend elements were created with the help of LLMs. However, the security elements were created mostly based off of class materials and extras.

Security Documentation:

- Index.php input type set to required to prevent empty prompts.
- POST used instead of GET for better security.
- Pdo used in database connection and for making prepared statements to prevent a layer of sql injection.
- Sql prevention in login.php->
    1. Generic session management
    2. Statements prepared with pdo to prevent sql injection.
    3. Input validation twice
    4. Password verification with hashes rather than plaintext to prevent mitm attacks.
    5. Error handling and generic error messages
    6. db.php used as header
- Very basic dashboard made, session check implemented, and logout.php to clear session
- Dashboard has username fetched, htmlspecialchars() used to prevent XSS (reflected).
- Session checks prevent unauthorized accesses.
- Added registration.php , added a registration link from index.php for easier access.
- Security measures in register.php:
    1. Mostly similar to login.php. Session tracking, prepared statements using pdo to prevent sql.
    2. There is an output here, so we use htmlspecialchars() used to prevent XSS (reflected).
    3. Unique username checked, passwords match checked.
    4. Automatic login after registration.
    5. Passwords are hashed using password_hash() and verified with password_verify(), preventing the use of plain text passwords.
- Document.cookie or document.location or any other untrusted dom not used, so the server is safe from XSS attacks (dom). All outputs are sanitized with htmlspecialchars() to prevent reflected XSS attacks.

- Made a sessioncheck.php, implemented as header for register.php and index.php , to be redirected to dashboard if someone tries to manual access through url, while being logged in. Done with session checks, and checking if its not dashboard.php, since it causes too many redirects.

Phase 2:

Decided to add a few more security features as an implementation of latest class materials and more.

- Added a text field to insert email in register.php. Email sanitization and validation carried out. Also checked if email is unique or not , otherwise 2fa hacks are possible. Redirects to index page when registration is successful.
- Login.php changed from echo for error messages to index.php redirect. Frontend purposes.
- Index.php now uses $error to give messages for users, frontend purposes. Outputs are sanitized with htmlspecialchars
- Reinforced alphanumeric only username in register.php
- Modified header sessioncheck to be more flexible in all of dashboard.php , register.php , index.php. Redirects to dashboard if logged in. Redirects to login page if logged out. All php files are in array and checked.
- Login.php updated. Now regenerates session after login, preventing session fixation attacks.
- Modified dashboard.php to silently unset session and log out after 30 seconds of inactivity. Timeout implemented for safety.
- Applied basic csrf protection to login.php, register.php and index.php.
- Implemented basic protection against dos attacks. Massive changes to login.php , now has rate limiting, session based. After 3 tries, exponential waiting time for wrong username or password for each wrong. Resets on successful login.
- Implemented a 2fa.php; loging.php now routes through 2fa.php , and then to dashboard.php. Direct url access has been blocked, modified through sessioncheck.php. 2fa has a timer of 3 minutes. Failure redirects to index.php. On confirmation from 2fa, dashboard is accessed. 2Fa is viewed through a debug echo , rather than email services , since it’s a local machine.